

ShopSphere

Project Architecture & Documentation

Platform	.NET 9 · ASP.NET Core Web API
Architecture	Clean Architecture + Microservices
Gateway	YARP Reverse Proxy
Auth	JWT · Refresh Tokens · BCrypt
Cache	Redis
Database	SQL Server
DevOps	Docker · docker-compose
Status	Phase 1 Complete → Phase 2 Active

Document Index

#	Section	Contents
1	Project Overview	Vision, tech stack, learning goals, system flow
2	Architecture	Microservices + Clean Architecture, folder structure
3	Infrastructure	SQL Server, Redis, Docker, YARP gateway
4	Identity Service	Auth system, endpoints, flows, caching
5	Roadmap	Phase-by-phase plan, future improvements

1. Project Overview

ShopSphere is an **enterprise-grade e-commerce backend platform** built to learn and implement production-level backend engineering — not basic CRUD development.

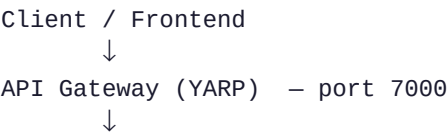
Tech Stack

Layer	Technology
Runtime	.NET 9
Web Framework	ASP.NET Core Web API
Architecture	Clean Architecture + Microservices
Database	SQL Server
Cache	Redis
Containers	Docker + docker-compose
API Gateway	YARP (Yet Another Reverse Proxy)
Auth	JWT — Access + Refresh Tokens
Logging	Serilog

Learning Goals

- Enterprise backend architecture patterns
- Scalable distributed systems design
- Production-ready authentication (JWT, refresh tokens, revocation)
- Containerized deployments with Docker
- API Gateway routing and reverse proxying
- Distributed caching with Redis
- Structured logging and observability
- Service-to-service communication
- CI/CD pipeline design
- Cloud-ready architecture

System Flow



■ Identity Service	■ live	■
■ Product Service	■ phase 2	■
■ Order Service	■ phase 3	■
■ Cart Service	■ phase 3+	■

Infrastructure

- SQL Server (host.docker.internal,1433)
- Redis (shopsphere-redis container)
- Docker Network

Running Endpoints

Access	URL
Identity Service direct	http://localhost:5000/swagger/index.html
Identity via Gateway	http://localhost:7000/identity/swagger/index.html

2. Architecture

ShopSphere uses **two complementary architectural styles** at different scales.

2.1 Microservices Architecture (Macro Level)

The system is divided into independent backend services. Each service owns its business logic, its database access layer, and can be deployed and scaled independently.

Service	Phase	Status
Identity Service	Phase 1	■ Complete
API Gateway (YARP)	Phase 1	■ Complete
Product Service	Phase 2	■ Next
Order Service	Phase 3	■ Planned
Cart Service	Phase 3+	■ Planned

2.2 Clean Architecture (Inside Each Service)

Each microservice is internally structured as four layers with strict one-directional dependencies.

Layer	Contains	Responsibility
API	Controllers, Middleware, Swagger, DI setup	HTTP handling, auth pipeline
Application	DTOs, Interfaces, Services, Use Cases, Validators	Business workflows, orchestration
Domain	Entities, Enums, Domain Models	Core rules — framework-free
Infrastructure	EF Core, Repos, Redis, JWT, Serilog, Integrations	DB, cache, external systems

Dependency Rule: API → Application → Domain. Infrastructure implements Application interfaces (Dependency Inversion). Nothing in Domain or Application knows about Infrastructure.

2.3 Updated Folder Structure

```
ShopSphere/
■■■■ ShopSphere.AllServices.sln          ← solution spanning all services
■
■■■■ services/
■ ■■■■ IdentityService/
■ ■ ■■■■ ShopSphere.IdentityService.sln ← per-service solution
■ ■ ■■■■ ShopSphere.IdentityService.API
■ ■ ■■■■ ShopSphere.IdentityService.Application
■ ■ ■■■■ ShopSphere.IdentityService.Domain
■ ■ ■■■■ ShopSphere.IdentityService.Infrastructure
■ ■
■ ■■■■ ShopSphere.ApiGateway
```

■	
■■■ docker/	← docker-compose.yml, infra setup
■■■ frontend/	← React web + admin (planned last)
■■■ docs/	← setup guides, API contracts
■■■ architecture/	← HLD/LLD diagrams, ADRs
■■■ .github/	
■■■ workflows/	← GitHub Actions CI/CD

Two Solution Files — Why?

File	Purpose
ShopSphere.AllServices.sln	Root solution — loads every service. Used for global operations, full builds, CI/CD.
IdentityService.sln	Per-service solution — only loads that service's 4 projects. Used for day-to-day development.

As new services are added (Product, Order, Cart), each gets its own .sln file and is also referenced in the root AllServices.sln.

3. Infrastructure

SQL Server

Primary persistent relational database. Stores users, products, orders, and all domain data. Each microservice owns its own schema following microservice data isolation principles.

Docker containers connect using:

```
host.docker.internal,1433
```

Redis

Distributed caching for fast-lookup storage. Reduces database load for frequently-read data.

Cache access pattern:

1. Request arrives
2. Check Redis for cached data
3. Cache hit → return from Redis (fast)
4. Cache miss → fetch SQL Server → populate Redis → return
5. On writes → update DB → invalidate / update cache

Docker

Container	Service	Exposes
shopsphere-api	Identity Service	8080 (internal)
shopsphere-gateway	YARP Gateway	7000 (external)
shopsphere-redis	Redis	6379 (internal)

```
# Start all containers
docker-compose up -d
```

API Gateway (YARP)

Single entry point for all clients. Reverse proxy that routes requests to the correct microservice. Clients never talk to services directly.

Gateway Path	Forwards To
/identity/*	http://shopsphere-api:8080
/product/*	TBD — Phase 2
/order/*	TBD — Phase 3

Planned Future Infrastructure

Component	Purpose	Phase
RabbitMQ / Kafka	Async event-driven messaging	4
Prometheus	Metrics collection	4

Component	Purpose	Phase
Grafana	Metrics dashboards	4
Elasticsearch + Kibana	Distributed log aggregation	4
OpenTelemetry	Distributed tracing across services	4
Kubernetes	Production container orchestration	5

4. Identity Service ■ Phase 1 Complete

The first completed microservice. Handles all authentication and user identity concerns.

Features

Feature	Status	Notes
User Registration	■ Done	With input validation
User Login	■ Done	Returns access + refresh token
Password Hashing	■ Done	BCrypt
JWT Auth	■ Done	Claims-based tokens
Refresh Tokens	■ Done	Rotated on use
Token Revocation	■ Done	Server-side revocation
Role Support	■ Done	Claims carry role info
Exception Middleware	■ Done	Centralized, consistent error format
Serilog Logging	■ Done	File, request, auth logs
Redis Caching	■ Done	Profile cache with invalidation

Folder Structure

```
IdentityService/
├── ShopSphere.IdentityService.API/
│   ├── Controllers/           ← AuthController, ProfileController
│   ├── Middleware/           ← ExceptionMiddleware
│   └── Program.cs             ← DI setup, pipeline config
├── ShopSphere.IdentityService.Application/
│   ├── DTOs/                 ← RegisterDto, LoginDto, TokenDto
│   ├── Interfaces/           ← IAuthService, ITokenService, ICacheService
│   └── Services/              ← AuthService, TokenService
├── ShopSphere.IdentityService.Domain/
│   ├── Entities/             ← User, RefreshToken
│   └── Enums/
├── ShopSphere.IdentityService.Infrastructure/
│   ├── Persistence/          ← AppDbContext, EF Migrations
│   ├── Repositories/
│   ├── Caching/              ← RedisCacheService
│   └── Auth/                  ← JwtTokenGenerator, PasswordHasher
```

Auth Flows

Registration:

POST /api/auth/register → validate → check email → BCrypt hash → save → 201

Login:

POST /api/auth/login → verify credentials → generate JWT + refresh token → 200

Refresh:

POST /api/auth/refresh → validate refresh token → new JWT → rotate refresh token

Revoke:

POST /api/auth/revoke → mark refresh token revoked in DB

Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	No	Register new user
POST	/api/auth/login	No	Login, receive token pair
POST	/api/auth/refresh	No	Refresh access token
POST	/api/auth/revoke	Yes	Revoke refresh token
GET	/api/profile	Yes	Get user profile (Redis cached)

5. Roadmap

Phase	Focus	Status
Phase 1	Foundation & Identity Service	■ Complete
Phase 2	Product Service	■ Next
Phase 3	Order Service	■ Planned
Phase 4	Advanced Backend Engineering	■ Planned
Phase 5	Cloud & DevOps	■ Planned
Frontend	React Web + Admin Dashboard	■ Last

Phase 2 – Product Service

- Product Service scaffold (API / Application / Domain / Infrastructure)
- Product entity & domain model (Name, Price, Stock, Category...)
- Product CRUD — Create, Read/List, Update, Delete
- Input validation with FluentValidation
- Redis caching for product reads
- Swagger / OpenAPI docs
- Dockerize Product Service & add to docker-compose
- YARP gateway route for /product/*

Phase 3 – Order Service

- Order Service scaffold
- Order creation workflow
- Inventory validation (calls Product Service via IHttpClientFactory)
- Service-to-service communication pattern
- Distributed transaction concepts — Saga pattern research

Phase 4 – Advanced Backend Engineering

- RabbitMQ or Kafka — event-driven async communication
- Health check endpoints (/health) on all services
- API versioning
- Distributed logging — ELK Stack (Elasticsearch, Logstash, Kibana)

- Prometheus metrics + Grafana dashboards
- OpenTelemetry distributed tracing

Phase 5 – Cloud & DevOps

- GitHub Actions CI pipeline (build + tests on PR)
- GitHub Actions CD pipeline (deploy on merge)
- Kubernetes local PoC with minikube
- Kubernetes deployment manifests (Pods, Services, Ingress)
- Azure or AWS deployment
- Production environment configuration

Frontend (Planned Last)

Frontend development is intentionally deferred until all backend services are stable and API contracts are finalized. This prevents costly UI rework caused by API changes.

Planned: React customer-facing web app (shopsphere-web) · React admin dashboard (shopsphere-admin)

Future Architecture Improvements

Pattern / Feature	Description
CQRS	Separate read/write models per service
Saga Pattern	Distributed transaction management
Service Discovery	Dynamic registry (e.g., Consul) — no hardcoded URLs
Centralized Auth Library	Shared NuGet package for auth concerns
Shared Building Blocks	Common Result types, pagination, error handling
API Rate Limiting	Throttle at gateway level
Feature Flags	Runtime feature toggling